

bcrypt - Guia de apresentacao e estudo

Seminario #01 - Tema 09 - Isabela Sousa e Paulo Victor Almeida

Material: bcrypt_slides_seminario.pdf (9 slides) + bcrypt_seminario.py

Este guia tem 3 partes: (1) divisao da apresentacao slide a slide e da demo ao vivo; (2) resumo para estudar COMO O CODIGO FUNCIONA, parte por parte; (3) perguntas provaveis com respostas e cuidados com afirmacoes que nao devem ser feitas.

Regra de ouro: a nota depende de apresentar E de responder perguntas. Cada um apresenta uma parte, mas os DOIS precisam saber o codigo inteiro.

PARTE 1 - Divisao da apresentacao (~12 a 13 min)

Slide	Quem	Tempo	Tema
1 - Capa	Isabela	0:30	Abertura e o problema (guardar senhas)
2 - O que e o bcrypt	Isabela	1:30	Hash de mao unica, sal, custo, anatomia do hash
3 - Os 6 passos	Paulo	2:00	Passos do algoritmo e a funcao de cada um no codigo
4 - Fluxo completo	Paulo	1:00	Diagrama: onde esta o loop 2^{custo}
5 - Exemplo pratico	Isabela	1:30	Senha 'senhaeu', custo 5, e como o login funciona
DEMO - codigo	Paulo narra / Isabela opera	3:00	Partes 1 a 5 do programa (ver abaixo)
6 - Por que e seguro	Paulo	1:00	Sal, custo, etapas em sequencia, tempo constante
7 - Vantagens e limites	Isabela	1:00	72 bytes, 4 KB, byte nulo
8 - bcrypt x scrypt x Argon2id	Paulo	1:00	Tabela comparativa
9 - Referencias	Isabela	0:30	Encerramento e convite a perguntas

Por que a demo vem depois do slide 5? O slide 5 mostra um exemplo real; ao rodar o codigo, a plateia ve os MESMOS passos acontecendo. Se faltar tempo, corte a Parte 4 da demo e deixe so o menu.

Falas sugeridas (resumo do que dizer em cada slide)

Slide 1 (Isabela)

Somos Isabela e Paulo Victor, tema 9: bcrypt. Todo site precisa guardar senhas, mas nunca a senha em si. A pergunta do nosso trabalho: como guardar do jeito certo? O hash que aparece na capa e um hash bcrypt real.

Slide 2 (Isabela)

bcrypt e uma funcao de hash feita para senhas, criada em 1999 por Provos e Mazieres, baseada na cifra Blowfish. Tres ideias: hash de mao unica (do hash nao se volta a senha), sal aleatorio de 16 bytes (mesma senha, hashes diferentes) e custo ajustavel (2^{custo} repeticoes; cada +1 dobra o tempo). Mostrar a anatomia: versao, custo, sal de 22 caracteres, hash de 31.

Slide 3 (Paulo)

Os 6 passos: (1) senha vira bytes + `\0`, corta em 72; (2) estado inicial com digitos de pi; (3) ExpandKey mistura sal e senha, 521 cifragens; (4) repete 2^{custo} vezes ExpandKey(senha) e ExpandKey(sal) - AQUI esta a lentidao; (5) cifra 64 vezes o texto fixo; (6) monta a string final. Cada caixa do slide tem embaixo o nome da funcao no codigo.

Slide 4 (Paulo)

Mesmo fluxo em diagrama. Destacar: o banco guarda so o hash; sal e custo ficam dentro dele, por isso o login consegue refazer o calculo. Com custo 5 sao 34.057 cifragens Blowfish.

Slide 5 (Isabela)

Valores reais do nosso programa para a senha 'senhaeu' com custo 5. Mostrar como P[0] muda a cada etapa (243F6A88 -> 74F1E7CD -> 03CCDF0A). No login: le custo e sal do hash, refaz o calculo com a senha digitada, compara em tempo constante. Uma letra diferente muda o hash por completo.

Slide 6 (Paulo)

Seguranca: o bcrypt nao impede o palpite, torna cada palpite caro. Sal unico (rainbow tables inuteis), custo acompanha o hardware (recomendado 12, minimo 10), etapas em sequencia (nao paraleliza), comparacao em tempo constante (evita timing attack). Os numeros de tentativas/s sao de uma fonte externa (Medium, 2025) - citar como tal.

Slide 7 (Isabela)

Vantagens: feito para senhas, sal embutido, custo ajustavel, usado desde 1999. Limitacoes: 72 bytes (o resto e ignorado - mostramos na demo), so ~4 KB de memoria (GPU/ASIC ajudam o atacante), byte nulo proibido, senha fraca continua fraca.

Slide 8 (Paulo)

bcrypt (1999, 4 KB fixa) x scrypt (2009, memoria configuravel) x Argon2id (2015, tempo/memoria/threads, recomendado pela OWASP). Para sistemas novos, Argon2id; bcrypt continua aceito (custo 10+).

Slide 9 (Isabela)

Referencias e codigo. Agradecer e abrir para perguntas.

Roteiro da demo ao vivo (python bcrypt_seminario.py)

Parte	O que aparece	O que dizer	Quem
1	P[0]=243F6A88 e primeira cifragem de bloco	Os digitos de pi sao calculados, nao colados, e batem com o paper do Blowfish. Uma rodada de Feistel mostrada.	Paulo
2	Os 6 passos com custo 4; contagem de cifragens	Cada [Passo N] e um passo do slide 3. 17.385 cifragens = (1+2*16)*521 + 192.	Paulo
3	Vetores oficiais: OK OK OK	Nosso codigo gera o MESMO hash que o bcrypt oficial: prova de que esta correto.	Isabela
4	Dois hashes da mesma senha; login; ataque	Sal: hashes diferentes. Ataque de dicionario acha 'senha123' na 6a tentativa: bcrypt nao salva senha fraca.	Isabela
5	Hashes iguais: True	Limite de 72 bytes: tudo depois do byte 72 e ignorado.	Paulo
Menu 1	Gerar hash ao vivo	Pedir uma senha ao professor/plateia e gerar com custo 4.	Paulo opera
Menu 2	Verificar senha	Colar o hash gerado: certa = CORRETA; mudar uma letra = INCORRETA.	Isabela opera
Menu 3	Atacar sua senha	Senha forte nao esta no dicionario; 'dragon' cai.	Paulo opera

Dicas: abra o terminal ANTES e rode uma vez para ver o tempo (cerca de 15 s ate o menu). Use custo 4 na demo (Python puro e lento; custo 8 leva varios segundos). Se der branco, diga: 'o esforco e medido em cifragens Blowfish; custo +1 dobra'.

PARTE 2 - Como o codigo funciona (guia de estudo)

O arquivo bcrypt_seminario.py tem 5 blocos (A a E) mais a demonstracao. Estude nessa ordem: cada bloco usa o anterior.

A	digitos de pi	-> P0, S0 (valores iniciais)
B	cifra Blowfish	-> Estado, funcao_f, cifrar_bloco, expandir_chave
C	base64 do bcrypt	-> b64_codificar, b64_decodificar
D	sal aleatorio	-> gerar_sal (SplitMix64)
E	bcrypt	-> hash_bruto, gerar_hash, verificar_senha
	demo	-> parte_1 .. parte_5, ataque, menu

0. Truques de Python usados

- M32 = 0xFFFFFFFF: 'x & M32' mantem so 32 bits (equivale a modulo 2^32). O Blowfish usa

palavras de 32 bits, mas o Python tem inteiros ilimitados.

- ^ e XOR (ou exclusivo). >> e deslocamento de bits. (x >> 24) pega o byte mais alto de um numero de 32 bits.
- int.from_bytes(b, 'big') junta bytes em um numero (big-endian: byte mais significativo primeiro).
- contador = [0] e uma lista de 1 posicao para poder alterar o valor de dentro das funcoes (sem 'global'). Conta cifragens de bloco.

A. Digitos de pi (arctan_inverso, tabelas_de_pi)

- O Blowfish precisa de $18 + 4 \times 256 = 1042$ numeros de 32 bits iniciais. Em vez de colar a tabela, calcula-se pi e usam-se os digitos depois da virgula.
- Formula de Machin: $\pi = 16 \cdot \arctan(1/5) - 4 \cdot \arctan(1/239)$. $\arctan(1/x)$ e calculado pela serie de Taylor: $1/x - 1/(3x^3) + 1/(5x^5) - \dots$
- Como nao ha decimais, tudo e multiplicado por uma 'escala' enorme ($1 \ll (\text{bits} + 64)$) e as divisoes sao inteiras (/). Os 64 bits extras evitam erro de arredondamento.
- 'pi - 3*escala' remove a parte inteira (3). A fracao e fatiada em palavras de 32 bits: as 18 primeiras formam P, as seguintes formam as 4 S-boxes de 256.
- Prova de que deu certo: P[0] = 243F6A88 e S0[0] = D1310BA6, iguais ao paper do Blowfish (parte_1 confere isso).
- Por que pi? 'Nothing up my sleeve': ninguem pode dizer que o autor escondeu uma backdoor nos numeros.

B. Blowfish

- Estado = vetor P (18 subchaves) + 4 S-boxes (cada uma 256 numeros). Comeca como COPIA dos digitos de pi e vai sendo reescrito pela senha e pelo sal.
- funcao_f(S, x): quebra x em 4 bytes a,b,c,d e calcula $((S0[a] + S1[b]) \text{ XOR } S2[c]) + S3[d]$, somas mod 2^{32} . Mudar 1 bit de x muda o resultado todo (efeito avalanche).
- cifrar_bloco(est, esq, dir_): rede de Feistel com 16 rodadas. Cada rodada: $\text{esq} \wedge= P[i]$; $\text{dir}_ \wedge= F(\text{esq})$; troca as metades. No fim desfaz a ultima troca e faz $\text{esq} \wedge= P[17]$, $\text{dir}_ \wedge= P[16]$.
- Exemplo da propria demo (bloco zerado, rodada 1): $\text{esq} = 0 \wedge P[0] = 243F6A88$; $F(243F6A88) = DB2F9C4E$; $\text{dir}_ = 0 \wedge DB2F9C4E$; troca -> $E = DB2F9C4E$, $D = 243F6A88$. Bate com a linha 'rodada 1' impressa.
- Estrutura de Feistel: e reversivel mesmo que F nao seja (por isso so precisa-se da subchave em ordem inversa para decifrar). No bcrypt so se CIFRA; nunca se decifra.

expandir_chave(est, chave, sal=None) - a operacao central

- Passo 1: $P[i] \wedge=$ palavra da chave, para i de 0 a 17. A chave e lida de 4 em 4 bytes dando a volta ($\text{pos} = (\text{pos} + 1) \% \text{len}(\text{chave})$) - chaves curtas sao repetidas ate cobrir os 72 bytes de P.
- Passo 2: comeca com bloco (0,0). Cifra -> resultado vira P[0],P[1]. Cifra o resultado -> P[2],P[3]... 9 cifragens para o P; depois 4 S-boxes x 128 blocos = 512 cifragens. Total $9 + 512 = 521$.
- Sal: se existir, e dividido em 4 palavras de 32 bits (ws). Antes de CADA cifragem, $\text{esq} \wedge= \text{ws}[k\%4]$ e $\text{dir}_ \wedge= \text{ws}[(k+1)\%4]$, com k andando de 2 em 2. Sem sal (sal=None) e o Blowfish normal.
- Dependencia em cadeia: cada cifragem usa o estado ja alterado e a saida da anterior. Por isso nao ha atalho nem paralelismo.

C. Base64 do bcrypt

- Converte bytes em texto. Alfabeto proprio './A-Za-z0-9' (diferente do base64 padrao) e sem '=' no final.
- 3 bytes (24 bits) = 4 caracteres de 6 bits. 16 bytes do sal = 5 grupos de 3 (20 chars) + 1 byte sobrando (2 chars) = 22 caracteres. 23 bytes do hash = 7 grupos (28 chars) + 2 bytes sobrando (3 chars) = 31 caracteres.
- Total do hash: $4 (\$2b\$) + 2 (\text{custo}) + 1 (\$) + 22 + 31 = 60$ caracteres.

- b64_decodificar faz o inverso, usando ALFABETO.index(c) para achar o valor 0..63 de cada caractere. E usada para recuperar sal e hash a partir da string do banco.

D. Sal (gerar_sal)

- Sem imports nao existe os.urandom. Usa-se o SplitMix64: a cada chamada soma uma constante ao estado e embaralha bits com multiplicacoes e XOR-shifts, gerando 8 bytes. Pega-se 16.
- A semente vem de enderecos de memoria (id()). E suficiente para demonstrar que cada hash recebe um sal diferente, mas NAO e criptograficamente seguro. Em producao: os.urandom / secrets.

E. O algoritmo: hash_bruto, gerar_hash, verificar_senha

```
hash_bruto(senha, custo, sal):
    chave = (senha + b'\x00')[:72]          # passo 1
    est = Estado()                          # passo 2 (copia de pi)
    expandir_chave(est, chave, sal)          # passo 3
    repita 2**custo vezes:                  # passo 4 (lentidao)
        expandir_chave(est, chave)
        expandir_chave(est, sal)
    cifra 64x os 3 blocos de 'OrpheanBeholderScryDoubt' # passo 5
    devolve os 23 primeiros dos 24 bytes
gerar_hash: '$2b$' + custo(2 digitos) + '$' + b64(sal) + b64(23 bytes) # passo 6
```

- Validacoes: custo entre 4 e 31, sal com 16 bytes, senha sem byte nulo (o \0 e o terminador que se acrescenta).
- Limite de 72: 18 palavras de P x 4 bytes = 72. O corte [:72] descarta o excedente, por isso duas senhas iguais nos 72 primeiros bytes geram o mesmo hash (parte_5).
- Passo 4 usa o proprio sal como 'chave' no segundo expandir_chave (16 bytes repetidos ciclicamente). Sem sal no argumento.
- O texto fixo tem 24 bytes = 3 blocos de 64 bits (esq,dir_). Cifrar 64 vezes cada bloco e o 'embaralhamento final'. Saida: 24 bytes, o ultimo e descartado -> 23 bytes. (Detalhe historico do bcrypt.)
- verificar_senha: separa '\$2b\$05\$...' em versao, custo e resto (53 chars). Os 22 primeiros viram o sal, os 31 ultimos o hash esperado. Recalcula com a senha digitada e compara. Nunca 'descriptografa'.
- Comparacao em tempo constante: dif |= x ^ y percorre TODOS os bytes e so no fim testa dif == 0. Um '==' para no primeiro byte diferente, e o tempo vazaria quantos bytes acertou (timing attack).

Numeros para decorar

Custo	Iteracoes	Cifragens Blowfish
4	16	17.385
5	32	34.057 (usado no slide 5)
12	4096	cerca de 4,3 milhoes

Formula: cifragens = (1 + 2*2^custo) * 521 + 192. Os 521 sao de cada ExpandKey (9+512); o 1 e o ExpandKey(sal,senha) inicial; os 2*2^custo sao os dois por iteracao; os 192 = 64 repeticoes x 3 blocos do final. Conferindo custo 5: 65*521 = 33.865; + 192 = 34.057.

Demonstracao (parte_1 a parte_5, ataque, menu)

- parte_1: confere pi com o paper e mostra 3 rodadas de Feistel.
- parte_2: gera hash de 'abc' com custo 4 e confere a contagem de cifragens com a formula.
- parte_3: 3 vetores (senha, hash) do bcrypt oficial; recupera o sal do hash esperado, gera com o nosso codigo e compara a partir do indice 4 (ignora \$2a\$ vs \$2b\$).
- parte_4: dois hashes da mesma senha (sais diferentes), login certo/errado e ataque de dicionario (verifica cada palavra da lista contra o hash).

- parte_5: duas senhas com 72 'A' iguais e finais diferentes -> hashes iguais.
- menu: 1 gera hash passo a passo; 2 verifica senha; 3 ataca sua senha; 0 sai.

Mapa: slide -> código

Slide	Onde esta no código
3 - passo 1	hash_bruto: chave = (senha + b'\x00')[:72]
3 - passo 2	class Estado e tabelas_de_pi (P0, S0)
3 - passo 3	expandir_chave(est, chave, sal) dentro de hash_bruto
3 - passo 4	for _ in range(2 ** custo) em hash_bruto
3 - passo 5	bloco 'blocos = ...' e 64 cifragens em hash_bruto
3 - passo 6	gerar_hash (f-string com b64_codificar)
6 - tempo constante	dif = x ^ y em verificar_senha
7 - 72 bytes	[:72] em hash_bruto; demonstrado em parte_5
5 - login	verificar_senha

PARTE 3 - Perguntas prováveis

P: O que é bcrypt?

R: Função de hash de senhas (1999) baseada no Blowfish. Lenta de propósito, com sal e custo ajustável.

P: Por que não SHA-256 para senhas?

R: SHA-256 é rápido: GPUs testam bilhões de palpites/s. O bcrypt encarece cada palpite.

P: O que é o sal e onde fica?

R: 16 bytes aleatórios por senha, guardados dentro do próprio hash (22 chars). Não é segredo; impede rainbow tables e esconde senhas repetidas.

P: O que é o custo?

R: Número N: o trabalho é repetido 2^N vezes. Cada +1 dobra o tempo. Recomendado 12 (mínimo 10).

P: Onde está a lentidão no código?

R: No laço for _ in range(2 ** custo) de hash_bruto: dois expandir_chave por volta, 521 cifragens cada.

P: Por que 521 cifragens por ExpandKey?

R: 9 para reescrever P (18 palavras) + 512 para as 4 S-boxes (4 x 256 palavras / 2 palavras por bloco).

P: O que é uma rede de Feistel?

R: Divide o bloco em duas metades e, em várias rodadas, mistura uma com a outra usando uma função F e uma subchave. Aqui: 16 rodadas, bloco de 64 bits.

P: De onde vêm os números iniciais do Blowfish?

R: Dos dígitos hexadecimais de pi. Calculamos com a fórmula de Machin. Garante que não há backdoor.

P: Por que o limite de 72 bytes?

R: O vetor P tem 18 palavras de 4 bytes = 72 bytes; a chave (senha + \0) é cortada nesse tamanho. O excedente é ignorado.

P: O que acontece com senhas > 72 bytes?

R: Só os 72 primeiros contam. Mostramos na parte_5: duas senhas diferentes depois do byte 72 geram o mesmo hash. Solução: pre-hash.

P: Por que o byte nulo é proibido?

R: Porque o programa acrescenta um \0 como terminador da chave; um \0 dentro da senha causaria ambiguidade.

P: Como o login verifica a senha?

R: Le custo e sal do hash guardado, recalcula com a senha digitada e compara os 23 bytes. Nao existe 'descriptografar'.

P: Por que comparar em tempo constante?

R: Um == para no primeiro byte diferente; medindo o tempo, o atacante descobriria quantos bytes acertou (timing attack).

P: Por que o texto 'OrpheanBeholderScryDoubt'?

R: E um texto fixo de 24 bytes (3 blocos de 64 bits) cifrado 64 vezes. Foi escolhido pelos autores; o conteudo em si nao importa, so que seja fixo e conhecido. (Nao afirmar a origem do texto.)

P: Por que descarta o ultimo byte?

R: Sao 24 bytes cifrados, mas o formato usa 23 (31 caracteres base64). Detalhe historico da especificacao.

P: O sal gerado no seu codigo e seguro?

R: Nao para producao: SplitMix64 semeado com enderecos de memoria. E so para a demo, pois nao pode importar os.urandom. Em sistemas reais usa-se o gerador do SO.

P: Como sabem que a implementacao esta correta?

R: Parte 3: para tres vetores do bcrypt oficial (incluindo o classico 'U*U'), geramos exatamente o mesmo hash.

P: Qual a diferenca entre \$2a\$, \$2b\$ e \$2y\$?

R: Versoes/correcoes de bugs. Para senhas curtas o hash e igual; so muda o prefixo. Por isso na parte_3 comparamos a partir do indice 4.

P: bcrypt e criptografia?

R: E hash de senhas. Usa uma cifra (Blowfish) por dentro, mas so cifra, nunca decifra. Nao e reversivel.

P: bcrypt x scrypt x Argon2id?

R: bcrypt usa ~4 KB fixos. scrypt e Argon2id exigem memoria configuravel, resistindo melhor a GPU/ASIC. OWASP recomenda Argon2id para sistemas novos; bcrypt segue aceito.

P: Se a senha for '123456', o bcrypt protege?

R: Nao. Ela cai em qualquer dicionario (mostramos na parte_4). O bcrypt so encarece cada palpite.

P: Por que a demo usa custo 4 e nao 12?

R: Python puro e muito mais lento que C. Custo 12 seria ~256x mais trabalho que o 4. Em producao usa-se 12 ou mais.

P: E se o professor pedir para mudar algo?

R: Custo: mudar CUSTO = 4 no topo da demo. Ver mais rodadas: trocar 'i < 3' por 'i < 16' em cifrar_bloco. Hash de uma senha especifica: menu opcao 1.

Cuidados: o que NAO afirmar

- NAO diga que 'OrpheanBeholderScryDoubt' e citacao de Dante: o comentario do codigo diz isso, mas nao conseguimos confirmar a origem. Diga apenas que e um texto fixo escolhido pelos autores.
- Os numeros do slide 6 (custo 6 ~500 mil tentativas/s; custo 12 ~8.300/s) vem do artigo do Medium citado (L. Lima, 2025). Diga a fonte; nao sao medidas do nosso programa. Se perguntarem, a ideia e que cada +1 de custo dobra o trabalho (6 -> 12 = 64x menos tentativas/s).
- O sal do nosso codigo nao e criptograficamente seguro (ver pergunta acima). Admitir isso e uma boa resposta.
- O cabeçalho do codigo diz 'COMO RODAR: python bcrypt_didatico.py', mas o arquivo se chama bcrypt_seminario.py. Rode pelo nome correto do arquivo (e, se quiserem, corrijam esse comentario).
- Tempo: Python puro e lento. Teste a demo antes e nao use custo acima de 6 ao vivo.

Checklist final

- ☐ Enviar bcrypt_seminario.py e o video no Classroom ate as 19h do dia da apresentacao.
- ☐ Abrir slides e terminal (na pasta do arquivo) antes de comecar.
- ☐ Cada um treina as suas falas em voz alta, cronometrando (~12 min).
- ☐ Os dois leem a Parte 2 e as perguntas da Parte 3.
- ☐ Combinar quem responde primeiro cada tipo de pergunta: codigo = quem apresentou a demo; teoria/seguranca = quem apresentou os slides 6 a 8.